

PCS-302 : Data Structures Lab

Queue Operations Using the Minimum Number of Stacks

Problem Statement

Design an algorithm and write a program to implement queue operations using the minimum number of stacks.

Minimum Number of Stacks Needed = 2

Unlike a queue (which could rotate itself to act like a stack), a single stack has no way to reverse the order of its own elements using only push/pop — a second stack is needed to temporarily hold elements while reversing them.

Example

```
enqueue(10)  --> Stack1 (in): 10
enqueue(20)  --> Stack1 (in): 10 20
enqueue(30)  --> Stack1 (in): 10 20 30

dequeue()    --> Stack2 (out) is empty, so transfer all of Stack1 into Stack2:
                Stack2 (top to bottom): 10 20 30
                pop Stack2 -> 10  (correct, oldest element first)
                Stack1: empty, Stack2: 20 30

enqueue(40)  --> Stack1 (in): 40

dequeue()    --> Stack2 not empty, just pop it -> 20  (correct)
```

Algorithm (Using 2 Stacks — "in" and "out")

```
Enqueue(k):
    push k onto Stack1 (in-stack)

Dequeue():
    if Stack2 (out-stack) is empty:
        while Stack1 is not empty:
            x = pop from Stack1
            push x onto Stack2
    if Stack2 is still empty: print "Underflow", return
    return pop from Stack2          // this is the front of the queue
```

Every element moves from Stack1 to Stack2 exactly once, so even though it looks like extra work, each element costs $O(1)$ on average (amortized).

C Program

```
#include <stdio.h>
#define MAX 10

int stack1[MAX], top1 = -1;    // in-stack (for enqueue)
int stack2[MAX], top2 = -1;    // out-stack (for dequeue)
```

```

int isEmpty1() { return (top1 == -1); }
int isEmpty2() { return (top2 == -1); }

void push1(int k) {
    top1 = top1 + 1;
    stack1[top1] = k;
}

int pop1() {
    int val = stack1[top1];
    top1 = top1 - 1;
    return val;
}

void push2(int k) {
    top2 = top2 + 1;
    stack2[top2] = k;
}

int pop2() {
    int val = stack2[top2];
    top2 = top2 - 1;
    return val;
}

int isEmptyQueue() {
    return (isEmpty1() == 1 && isEmpty2() == 1);
}

void enqueue(int k) {
    if (top1 == MAX - 1) {
        printf("Overflow. Cannot enqueue %d.\n", k);
        return;
    }
    push1(k);
    printf("%d enqueued to queue.\n", k);
}

void dequeue() {
    if (isEmpty2() == 1) {
        // move everything from stack1 into stack2 (reverses the order)
        while (isEmpty1() == 0) {
            int val = pop1();
            push2(val);
        }
    }
    if (isEmpty2() == 1) {
        printf("Queue Underflow.\n");
        return;
    }
    int val = pop2();
    printf("%d dequeued from queue.\n", val);
}

void printQueue() {

```

```

    int i;
    if (isEmptyQueue() == 1) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue (front to rear): ");
    for (i = top2; i >= 0; i--)
        printf("%d ", stack2[i]);
    for (i = 0; i <= top1; i++)
        printf("%d ", stack1[i]);
    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    printQueue();

    dequeue();
    enqueue(40);
    printQueue();

    dequeue();
    printQueue();

    return 0;
}

```